
1. **synchronized** (Thread Synchronization)

Use Case: Prevents multiple threads from accessing a critical section simultaneously.

Types of Synchronization

1. Method-level Synchronization

```
class SharedResource {
    public synchronized void printNumbers() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " - " + i);
        }
    }
}

public class SynchronizedExample {
    public static void main(String[] args) {
        SharedResource resource = new SharedResource();

        Runnable task = () -> resource.printNumbers();

        new Thread(task).start();
        new Thread(task).start();
    }
}
```

- ◆ Locks the entire method, allowing only one thread to execute at a time.

2. Block-level Synchronization

```
class SharedResource {  
    public void printNumbers() {  
        synchronized (this) { // Synchronizing a specific block  
            for (int i = 1; i <= 5; i++) {  
                System.out.println(Thread.currentThread().getName() + " - " + i);  
            }  
        }  
    }  
}
```

- ◆ More fine-grained control than method-level synchronization.
-

2. **volatile** (Visibility Control)

Use Case: Ensures that changes to a variable are immediately visible to all threads.

```
class VolatileExample {
    private static volatile boolean running = true;

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            while (running) { /* Running */ }
            System.out.println("Thread stopped.");
        });

        t1.start();

        try { Thread.sleep(2000); } catch (InterruptedException e) { }

        running = false; // Change is visible to all threads
        System.out.println("Flag set to false.");
    }
}
```

- ◆ Prevents CPU caching optimizations but does NOT prevent race conditions.
 - ◆ If multiple threads modify the variable, use **Atomic** types or locks.
-

3. Atomic Data Types (**AtomicInteger**, **AtomicBoolean**, ..)

Use Case: Provides thread-safe operations without using locks.

```
import java.util.concurrent.atomic.AtomicInteger;

public class AtomicExample {
    private static AtomicInteger count = new AtomicInteger(0);

    public static void main(String[] args) {
        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) {
                count.incrementAndGet(); // Thread-safe increment
            }
        };

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) { }

        System.out.println("Final count: " + count.get());
    }
}
```

- ◆ No need for **synchronized** or locks, making it more efficient.
 - ◆ Supports atomic operations like **incrementAndGet()**, **compareAndSet()**, etc.
-

4. ReentrantLock (Explicit Locking)

Use Case: Provides fine-grained locking with more features than **synchronized**.

```
import java.util.concurrent.locks.ReentrantLock;

public class ReentrantLockExample {
    private static final ReentrantLock lock = new ReentrantLock();

    public static void main(String[] args) {
        Runnable task = () -> {
            lock.lock();
            try {
                System.out.println(Thread.currentThread().getName() + " acquired lock");
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                lock.unlock();
                System.out.println(Thread.currentThread().getName() + " released lock");
            }
        };

        new Thread(task).start();
        new Thread(task).start();
    }
}
```

♦ Advantages over **synchronized**:

- Can use **tryLock()**
 - Allows **fair locking** (**new ReentrantLock(true)**)
 - Can be **interrupted** (**lockInterruptibly()**)
-

5. Semaphore (Controlling Access)

Use Case: Restricts the number of threads accessing a resource.

```
import java.util.concurrent.Semaphore;

public class SemaphoreExample {
    private static final Semaphore semaphore = new Semaphore(2); // Allows 2 threads

    public static void main(String[] args) {
        Runnable task = () -> {
            try {
                semaphore.acquire();
                System.out.println(Thread.currentThread().getName() + " acquired a pe
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                semaphore.release();
                System.out.println(Thread.currentThread().getName() + " released a pe
            }
        };

        for (int i = 0; i < 5; i++) {
            new Thread(task).start();
        }
    }
}
```

- ♦ **Use Case:** Limiting concurrent database connections, API requests, etc.
-

6. Mutex (Mutual Exclusion)

Use Case: Ensures exclusive access to a critical section.

```
import java.util.concurrent.locks.ReentrantLock;

public class MutexExample {
    private static final ReentrantLock mutex = new ReentrantLock();

    public static void main(String[] args) {
        Runnable task = () -> {
            mutex.lock();
            try {
                System.out.println(Thread.currentThread().getName() + " acquired mutex");
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                mutex.unlock();
                System.out.println(Thread.currentThread().getName() + " released mutex");
            }
        };

        new Thread(task).start();
        new Thread(task).start();
    }
}
```

7. Futures & CompletableFutures

Futures (Blocking)

```
import java.util.concurrent.*;

public class FutureExample {
    public static void main(String[] args) throws ExecutionException, InterruptedException {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Future<Integer> future = executor.submit(() -> {
            Thread.sleep(1000);
            return 42;
        });

        System.out.println("Result: " + future.get()); // Blocks until result is available
        executor.shutdown();
    }
}
```

CompletableFuture (Non-Blocking)

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureExample {
    public static void main(String[] args) {
        CompletableFuture.supplyAsync(() -> "Hello")
            .thenApply(str -> str + " World")
            .thenAccept(System.out::println);
    }
}
```

- ◆ Supports chaining & exception handling.
-

8. Executors & ThreadPoolExecutor

Using Executors (Thread Pool)

```
import java.util.concurrent.*;

public class ExecutorExample {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(3);

        Runnable task = () -> System.out.println(Thread.currentThread().getName() + "
");

        for (int i = 0; i < 5; i++) {
            executor.submit(task);
        }

        executor.shutdown();
    }
}
```

Using ThreadPoolExecutor (Custom Thread Pool)

```
import java.util.concurrent.*;

public class ThreadPoolExecutorExample {
    public static void main(String[] args) {
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            2, 4, 60, TimeUnit.SECONDS, new LinkedBlockingQueue<>(10)
        );

        for (int i = 0; i < 5; i++) {
            executor.execute(() -> System.out.println(Thread.currentThread().getName()));
        }

        executor.shutdown();
    }
}
```

- ◆ More control over thread pool behavior.
-